



AGENDA CULTURAL

COSTA RICA

PSWE-04: Proyecto Funcional

Daniel Canessa - Stephanie Delgado

Agenda



1. Introducción a la plataforma.
2. Hallazgos de la arquitectura de la información.
3. Casos de uso del MVP.
4. Is / Is not del MVP.
5. Arquitectura / Diseño / Tech Stack.
6. Demo.
7. Q/A

Agenda Cultural Costa Rica



Es una plataforma abierta para descubrir, organizar y visibilizar la cultura en Costa Rica.

Link: <https://github.com/danielcanessa/AgendaCulturalCostaRica/>

Objetivos:

- **Unificación** de un catálogo de eventos con búsqueda y filtrado.
- **Gestión sencilla** para la creación y administración de eventos, usuarios, agendas, organizaciones, etc priorizando el findability.
- **Plataforma accesible** con diseño orientado a WCAG 2.1 nivel AA y promoviendo que los eventos cuenten con información sobre accesibilidad.

Arquitectura de información

Problemas (evidenciados en el análisis de personas y el análisis competitivo)	Oportunidades (desarrolladas en el inventario de contenidos)
La información de eventos culturales está fragmentada (redes sociales, afiches, sitios web). Además, es difícil de buscar/filtrar .	Catálogo de eventos centralizado con filtros y ficha de evento estándar para todos los eventos.
En ningún sitio de eventos de Costa Rica se menciona la accesibilidad del evento (rampas, intérprete, asientos, etc.).	Hacer de la accesibilidad un dato de primer nivel en cada evento.
Ningún sitio web costarricense permite a usuarios publicar sus eventos , solo los administradores del sitio pueden publicar.	Democratizar la publicación de eventos y hacer que los administradores los moderen .
No existen proyectos abiertos para que Municipidades o ONGs difundan oferta cultural.	Utilizar una licencia abierta y exponer un API que pueda ser consumido desde otros sitios web.

Casos de uso del MVP

1. **Usuario no registrado descubre eventos filtrando** por **categoría** o **nombre** y **consulta** la ficha del **evento**, que incluye detalles y especifica la accesibilidad del evento.
2. Un **usuario registrado comenta** y **califica eventos**, mientras que un administrador modera los comentarios.
3. Un **usuario registrado guarda eventos** que le interesan en su agenda y luego los **consulta** desde su **perfil**.
4. Un **usuario** registrado **publica** un **evento** completando un formulario que le permite añadir un banner para el evento y seleccionar la ubicación en un mapa, al enviarlo, el **evento** queda **pendiente** de **aprobación** por parte de un administrador.
5. Un **administrador** maneja los **eventos** y decide **aprobar, solicitar cambios o rechazarlos**.
6. Un **usuario registrado** tiene acceso a su **perfil** desde el cual **puede editar** su **información**, **revisar** y **editar** sus **eventos** creados y **administrar** las **organizaciones que ha creado**.

Roles de usuario y acciones

Perfil	Descripción	Acciones principales
Público	Visitante sin cuenta	● Ver y filtrar eventos ● Abrir ficha que incluye datos de accesibilidad
Usuario registrado	Visitante con cuenta y logeado.	● Ver y filtrar eventos ● Abrir ficha que incluye datos de accesibilidad ● Comentar y calificar eventos ● Guardar eventos en la agenda. ● Gestionar su perfil / cuenta. ● Crear/editar/eliminar sus eventos.
Administrador	Responsable de la moderación	● Aprobar eventos ● Editar/eliminar contenidos cuando corresponda. ● Gestión de roles y cuentas.

Is / Is Not

Is:

- Catálogo de eventos con filtros por categoría y nombre.
- Creación/edición/eliminación de eventos por sus creadores o administradores.
- Interacción básica: comentarios y calificaciones por evento.
- Favoritos: guardar/consultar eventos.
- RBAC (role based access control): público, usuario registrado y administrador.
- MVP web abierto (licencia MIT) para descubrir y gestionar eventos culturales enfocado en Costa Rica.

Is Not:

- No es una pasarela de pagos, de venta de boletos o responsable de eventos.
- No es una aplicación de tiempo real (notificaciones, feed en vivo).
- No es una red social (no mensajes; comentarios acotados al evento).
- No hace scraping de sitios externos: los eventos se publican por formulario o API.
- No es un Sistema de Gestión de Contenidos (CMS) genérico.
- No es una app móvil nativa (single page application web “SPA”).
- Pruebas de usabilidad por usuarios reales.

Arquitectura Cliente–Servidor

- Cliente: single page application (SPA) desarrollado en React 19.1.0.
- Servidor: Django REST Framework (DRF) 4.2.14.
- Base de datos: MySQL 9.3.0.

Se prioriza el desacople y la adaptabilidad, para incorporar nuevos clientes (móvil / portales municipales) y además agregar nuevas funcionalidades.



Frontend:

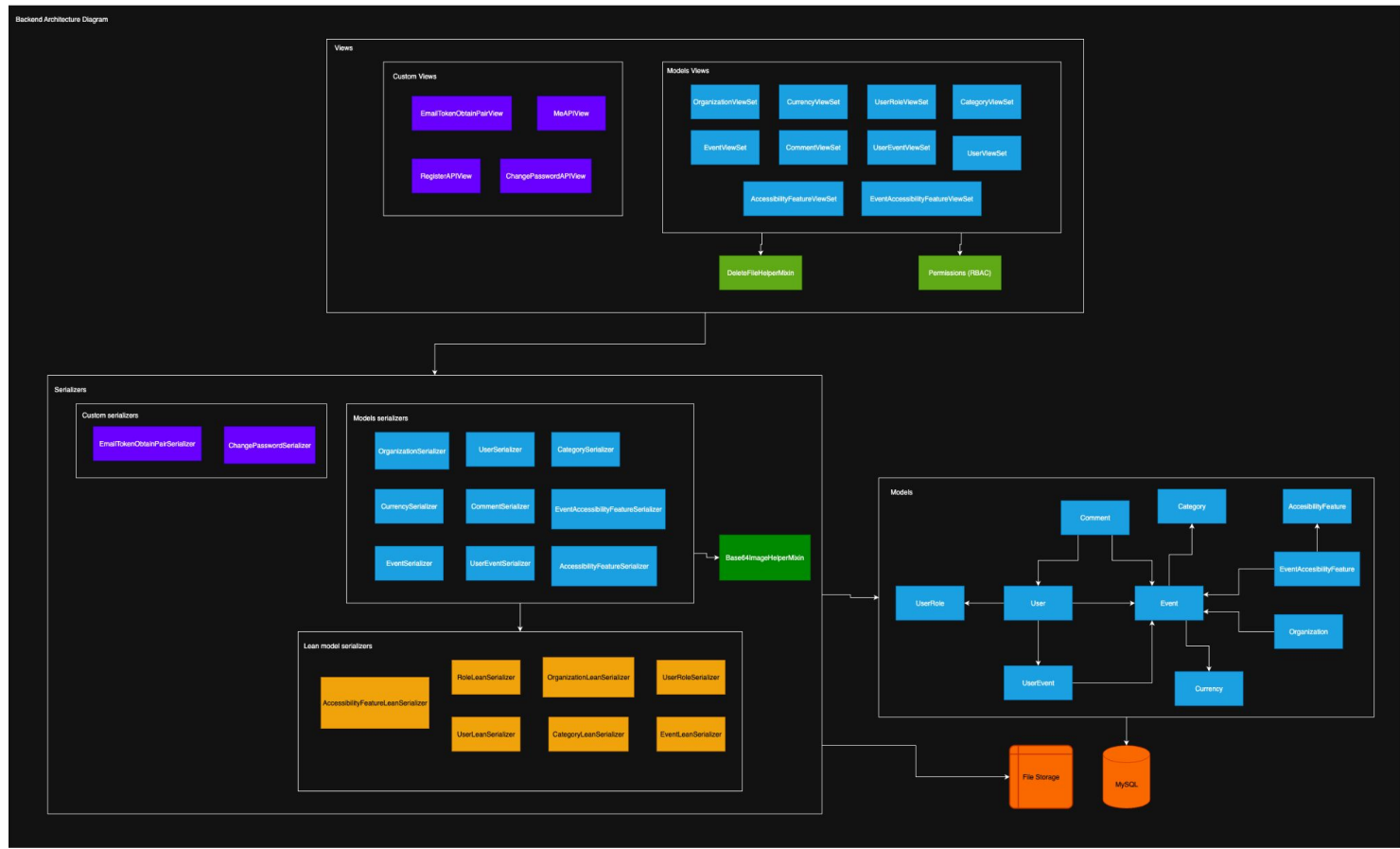
- Se **desarrolló** utilizando **React 19.1.0**.
- **Prioriza** una **navegación fluida** utilizando técnicas como **paginación**.
- **Diseño modular y escalable**.
 - Componentes reutilizables para garantizar coherencia visual y funcional.
- Utiliza **JSON Web Token (JWT)** para interacciones con el backend.
- **Accesibilidad**:
 - Principios **WCAG 2.1 nivel AA**.
 - Modo de **alto contraste**.
 - **Navegación por teclado** compatible con lectores de pantalla.
 - **Alt text** en todas las imágenes, formularios, etc.

Nota: el nivel AA de **WCAG** asegura que la plataforma sea usable por la mayoría de las personas, incluidas aquellas con discapacidades, es el más utilizado a nivel internacional.

Backend:

- Se **desarrolló** utilizando **Django 4.2.14**.
 - Aporta un **marco modular, extensible** y con **componentes reutilizables**.
- Arquitectura model-view-controller (**MVC**)
 - **Models**: definen la estructura y relaciones de datos.
 - i. Se utiliza Django como Object-Relational Mapper (ORM).
 - **Serializers** (views): funcionan como DTOs.
 - i. Transforman los modelos a JSON.
 - ii. Validan entradas, excluyendo campos sensibles (como password).
 - iii. Hacen el manejo de archivos (ejemplo imágenes base64 → file storage).
 - **ViewSet** (controllers): gestionan la lógica de negocio y el Role-Based Access Control (RBAC).
 - i. Manejo de filtros para requests de REST.
- **Seguridad**: autenticación con **JSON Web Tokens** (JWT) *access/refresh*, validación de permisos y autenticación en cada endpoint.

Diagrama arquitectura backend



Frontend	Backend	Base de datos	Seguridad
• React 19.1.0 — Aplicación de una sola página (<i>Single Page Application, SPA</i>) • React Router DOM 7.7.1 — Ruteo en el cliente • React Leaflet 5.0.0 — Mapas interactivos • Tailwind CSS 4.1.11 — Framework de estilos utilitario • Testing Library — Pruebas unitarias de componentes	• Django 4.2.14 — Framework web con patrón Modelo–Vista–Controlador (<i>Model–View–Controller, MVC</i>) • Django REST Framework (DRF) 3.16.0 — Construcción de APIs REST • Django Filter 24.2 — Filtrado dinámico de consultas • Django REST Password Reset $\geq 1.3.0$ — Recuperación de contraseñas vía API • Django CORS Headers 4.3.1 — Configuración de <i>Cross-Origin Resource Sharing</i> (CORS) • PyMySQL 1.1.1 — Conector para MySQL	MySQL 9.3.0 — Sistema de gestión de bases de datos relacional (<i>Relational Database Management System, RDBMS</i>)	• JSON Web Tokens (JWT) — Manejo de sesión <i>stateless</i> (paquete SimpleJWT 5.3.1) • Cryptography $\geq 3.3.2$ — Seguridad y cifrado

Demo

Preguntas